

What I Built, Level by Level

The whole system comes down to one rule: numbers can only come from SQL, everything else (context, disputes, delays) comes from chat. I built a ProvenanceLedger that tracks where every value came from, and the final answer step checks every number against it before it's allowed to reply. Everything below is either that rule being built out (Levels 0–2, all done) or the harder problems on top of it (Level 3 — some done, some not).

L0 The Core Problem

DONE

THE PROBLEM

A business has two sources of truth. The ERP is clean and reliable but always a bit behind. The team chat is current but messy and informal. Neither one alone gives the full picture, and mixing them carelessly is worse than using just one.

EXAMPLE

ERP says Sharma Fabrics owes ₹4,20,000. Only chat knows he's disputing ₹80,000 of it. If I only show the ERP number, it's not technically wrong, but it's misleading.

WHAT I DID

I made one hard rule: numbers only come from SQL, everything else comes from chat. Every tool result gets logged into a ProvenanceLedger the moment it's produced, and the final_answer step checks every number in the reply against that ledger before it's allowed to send.

How the pieces connect

ERP tables -> tagged into ProvenanceLedger

Chat search -> tagged into ProvenanceLedger

|
v

final_answer checks every number
against the ledger before replying

Level 1 — Getting Each Side Working On Its Own

1a. Pulling data from the ERP safely

DONE

THE PROBLEM

I needed to answer questions from the ERP tables, but I didn't want the AI writing its own SQL — that's unpredictable and risky.

EXAMPLE

"What's Sharma Fabrics' total outstanding?"

WHAT I DID

Wrote structured tools schemas which will be filled by LLM and we will create the query— get_customer_outstanding, get_supplier_pos, and so on. The AI only picks which tool to call and what to search for; it never sees or writes SQL.

Each tool's search field also checks the `trusted knowledge_base` table before falling back to a plain name match — so an already-confirmed alias like "supp B" resolves the same way whether the AI calls `resolve_entity` first or just passes the alias straight into `get_supplier_pos`.

1b. Searching chat by meaning

DONE

THE PROBLEM

Chat uses different words for the same thing — "supp B" vs "Supplier B Ltd." A normal keyword search misses this.

EXAMPLE

"What's been said about Supplier B?"

WHAT I DID

I embedded every message once, and at query time I embed the question and compare it using cosine similarity. No vector database — with only a few hundred messages per tenant, an in-memory search is fast enough and simpler to run.

Level 2 — Make Them Talk to Each Other

L2.1 Matching Chat Mentions to ERP Records

DONE

THE PROBLEM

Chat mentions things informally — "supp B", "4812" — and these have to map to the exact ERP record before I trust any number tied to them. A wrong match quietly breaks everything downstream.

EXAMPLE

"Supplier B" should map to SUP-1043. But SKU-99201 and SKU-99210 look almost identical and are completely different items.

WHAT I DID

- Exact match? Check `knowledge_base` for "supp B" literally. If it's there (already confirmed before), use it — resolves to SUP-1043 instantly.
- Close enough to something confirmed? If not exact, fuzzy-match against already-confirmed phrases only (never a raw ERP name). "supp B's" would still match "supp B" here.
- Nothing confirmed yet? Rank every candidate — ERP names/codes and confirmed aliases — by text or embedding similarity. This is how "the dye guys" can still match "ColorTech Dyers" despite sharing no letters.
- High score $\neq$ trusted. No matter the score, the AI asks a person to confirm before using it. Only a confirmed match gets written to `knowledge_base` — and gets embedded immediately, so future rewordings of it match faster too.

L2.2 Deciding Where to Look, Before Looking

DONE

THE PROBLEM

Before pulling any data, the AI needs to decide upfront: do I need SQL, chat, or both? Otherwise it just grabs whatever tool seems convenient in the moment.

EXAMPLE

"Why is order #4521 late?" needs SQL to confirm it's late, and chat to explain why (the dyeing line was down).

WHAT I DID

I added a `plan_query` step that's mandatory. Every SQL tool and the chat-search tool check whether a plan exists, and whether that plan actually says this source is needed. If the AI says "I don't need chat" and then tries to search chat anyway, it gets blocked, not silently allowed.

WORTH KNOWING

This makes sure the AI sticks to its own plan — it doesn't check whether the plan itself was the right call. That needs a proper test set, which is 3.6 below and still not built.

```
app/tools/base.py – the check every tool runs first
if REQUIRES_PLAN and no plan yet:
    block, say 'call plan_query first'
if this is a SQL tool but plan says needs_sql=false:
    block, say 'your plan said you don't need SQL'
if this is the chat tool but plan says needs_chat=false:
    block, say 'your plan said you don't need chat'
```

L2.3 The Hard Line: No Fake Numbers

DONE

THE PROBLEM

The AI should never put a number in the final answer unless it genuinely came from SQL.

EXAMPLE

It should never write ₹4,20,000 in an answer just because it saw that number sitting in a chat message.

WHAT I DID

Every number a tool returns gets logged into the ProvenanceLedger before the AI even sees it. When the AI tries to give a final answer, every number in it is checked against the ledger — if the reference doesn't exist, or the value doesn't match exactly, the answer is rejected and the AI has to try again.

WORTH KNOWING

This isn't a prompt asking the AI nicely not to make up numbers — it's a check the code runs before it lets the answer go out. The AI literally can't finish the turn with a made-up number.

```
app/tools/final_answer_tool.py – checked before every reply
for each number in the answer:
    entry = ledger.get(number.ref)
    if entry doesn't exist -> reject, 'you made this ref up'
    if entry.value != number.value -> reject, 'doesn't match the ledger'
```

Level 3 — The Genuinely Hard Part

This is where I'll be most honest. Some of this works, some is partial, and some I haven't built yet. Here's exactly where each one stands.

3.1 Chat and SQL Disagreeing With Each Other

NOT DONE YET

THE PROBLEM

ERP says PO-4812 is fully open. A chat message from Friday says it arrived short by 300 units. Right now, nothing checks the two against each other — I just have both pieces of data sitting separately.

EXAMPLE

This exact case is in my seed data (a chat message says "4812 partially received, 300 units short") but the system never flags that it contradicts the ERP status.

WHAT I'D BUILD

I'd add a check that compares what a chat message implies about an entity against its SQL status, and if they disagree, show both — with a fixed rule that SQL always drives the number and chat always drives the caveat, same as the hard line above.

3.2 A Chat Detail That Changes What a Number Means

PARTIAL

THE PROBLEM

₹80,000 of the ₹4,20,000 Sharma owes is disputed, according to chat. Should the answer show one number, two, or a number with a note attached?

EXAMPLE

The seed data has a message specifically for this: "Sharma disputing 80k, says short supply."

WHAT I DID

Right now the ₹80,000 can't sneak into the final answer as a fake fact — the hard line already blocks that. It can only show up as a plain sentence, not as a real, structured figure. What's missing is a proper "disputed amount" type with its own reference in the ledger, so it can be shown as an actual number instead of just prose.

3.3 Telling Silence Apart From Missing Data

NOT DONE YET

THE PROBLEM

If nobody's mentioned Supplier C in two months, is that good news or just missing data? Right now I can't tell the difference.

EXAMPLE

A chat search for a supplier that returns nothing looks exactly the same whether that supplier is doing fine or has simply never come up in chat.

WHAT I'D BUILD

I'd track the last time each entity was mentioned in chat. That way, 'no results' can mean 'discussed regularly, nothing wrong' instead of just 'never came up.'

3.4 Different Tenants, Different Words

PARTIAL

THE PROBLEM

One tenant says "lot", another says "batch". This has to work without me writing custom code for each tenant.

EXAMPLE

The same word can mean different things too — "line 2" could mean a machine for one tenant and a product line for another.

WHAT I DID

I built a knowledge_base table that already solves this for entity names — "supp B" maps to SUP-1043, and it's pure data, no code changes needed per tenant. What's missing is the same idea one level up, for word meaning: a table like {tenant_id, term, meaning}. Same pattern, just not built yet.

3.5 Knowing Where Every Number Came From

DONE

THE PROBLEM

For any number in an answer, I should be able to say exactly where it came from — not just "the AI said so."

EXAMPLE

"You said Sharma owes ₹4,20,000 — where's that from?" should have a real, checkable answer.

WHAT I DID

The ProvenanceLedger already does this — every value gets a reference, a source, and the exact raw value the moment it's created. It's the same mechanism that powers the hard line above, not a separate feature I bolted on.

3.6 Testing That Answers Stay Correct

NOT DONE YET

THE PROBLEM

I have no automatic way to check that answers stay correct after I change something — a prompt, a threshold, anything.

EXAMPLE

If I change TRUSTED_FUZZY_THRESHOLD — the one threshold left that decides trust — from 0.75 to 0.65, right now the only way to check it still works is trying it by hand.

WHAT I'D BUILD

This is where the agent evaluation framework comes in. There are many strategies which I am currently reading about.

What I'd Build Next

In this order, because each one makes the next one safer to build:

1. Testing (3.6)

First, because every other fix below changes behavior in ways I can currently only check by hand.

2. Chat vs. SQL contradiction check (3.1)

Once I have tests, this is safe to build without breaking things silently.

3. Real "disputed amount" values (3.2)

Turn things like the ₹80,000 dispute into an actual structured number, not just a sentence.

4. Last-mentioned tracking (3.3)

So silence in chat means something specific instead of nothing.

5. Term-meaning table (3.4)

Same pattern as the entity table, one level up, for words that mean different things per tenant.

One thing I deliberately didn't build: a full planner/DAG/multiple-agents setup. With only two data sources (SQL and chat), that felt like more complexity than the problem needed. If the number of sources grows, I'd revisit it.